VETERINARIA EIVET / DOCUMENTACIÓN

Manual de despliegue e instalación

Flutter Android + panel web + Supabase

Dirigido a
Responsables técnicos de instalación, operación y publicación de EIVET. No contiene contraseñas ni claves.

Edición: 10 de octubre de 2026

Versión del proyecto consultado: 3.0.0+1. Este documento explica el uso o el procedimiento; no certifica por sí mismo pruebas ni despliegues.

1. Alcance y arquitectura

EIVET usa una base de código Flutter: en Android muestra la aplicación del propietario y en web el panel administrativo. Ambos usan el mismo proyecto Supabase configurado para Auth, PostgreSQL con RLS, Storage y Edge Functions. El panel web se compila y publica en Netlify.

Componente	Punto de instalación o despliegue
Android	APK compilado desde Flutter. Paquete: bo.edu.uajms.proyecto_final_360.
Web	Compilación Flutter Web en build/web; alojamiento definido en netlify.toml.
Supabase	Auth, tablas SQL, políticas RLS, RPC salud y funciones salud / crear-propietario.

Estado que debe comprobarse
Los archivos del repositorio describen el esquema y la publicación previstos; no prueban que la base, RLS, funciones y APK final estén sincronizados en producción. Registra la versión, fecha y resultado de cada verificación.

Herramientas y accesos

Se necesitan Flutter (el proyecto exige Flutter 3.35 o superior y Dart 3.9), Android SDK, Java 17 para compilar Android, acceso autorizado al proyecto Supabase y a su SQL Editor, y acceso al sitio Netlify/Git para publicar web. El build de Netlify fija Flutter 3.44.8.

2. Preparación local

01 Trabaja con la revisión Git acordada y registra su commit. Comprueba el entorno con **flutter doctor**.

02 Copia **.env.example** a **.env**. Define APP_MODE=supabase, SUPABASE_URL y SUPABASE_PUBLISHABLE_KEY del mismo proyecto Supabase.

03 Ejecuta **flutter pub get**. Para desarrollo local usa **flutter run --dart-define-from-file=.env** con un Android conectado o un emulador.

04 Mantén .env fuera de Git y de las capturas. La publishable key puede estar en el cliente; una clave service_role o sb_secret_ nunca debe entrar en Flutter, Netlify público ni Postman.

```
Copy-Item .env.example .env
# Completar .env localmente, sin publicar sus valores
flutter pub get
flutter run --dart-define-from-file=.env
```

3. Preparar Supabase

Entorno nuevo: ejecuta los SQL desde Supabase SQL Editor en el orden documentado. **Producción existente:** antes de aplicar cualquier script, respalda y compara el esquema y políticas activos. No vuelvas a ejecutar toda la secuencia a ciegas.

Orden	Archivo SQL
1	01_SCHEMA_RLS.sql
2	02_MASCOTAS_CRUD_RLS.sql
3	05_ADMIN_WEB_SCHEMA_RLS.sql
4	06_VINCULAR_CITAS_CLIENTES.sql
5	09_TRATAMIENTOS_CLIENTES_RLS.sql
6	10_CLIENTES_EN_PANEL.sql
7	12_MASCOTAS_TITULAR_SOLO_PERSONAL.sql
8	13_CITAS_RLS_PROPIETARIOS.sql

Ejecuta también **supabase/salud.sql** para crear la RPC de salud. El script 04_DATOS_EJEMPLO_CUIDADOS.sql es opcional y requiere cuentas/mascotas de ejemplo. Comprueba en Supabase que existan las tablas y que las políticas RLS estén activas, especialmente las de mascotas y citas.

4. Usuarios, roles y funciones

Cuentas y roles

Una cuenta de Auth recibe un UUID. El trigger crear_cliente crea su fila en clientes. Para personal, crea la cuenta con acceso administrativo a Supabase y una fila en public.perfiles con el mismo UUID y rol administrador o veterinario. El panel web consulta ese rol al iniciar sesión. La función crear-propietario solo crea cuentas de propietario si llama un administrador.

Desplegar Edge Functions

01 Instala Supabase CLI, inicia sesión con una cuenta autorizada y enlaza este repositorio con el proyecto correcto.

02 Despliega salud y crear-propietario desde la raíz del proyecto. Verifica sus respuestas en el entorno de destino.

03 La configuración local marca verify_jwt=false para ambas funciones: salud es pública; crear-propietario valida dentro del código el JWT y el rol administrador. La clave de servicio se mantiene solo en Supabase.

```
supabase login
supabase link --project-ref PROJECT_REF
supabase functions deploy salud
supabase functions deploy crear-propietario
```

Verificación de publicación
GET /functions/v1/salud debe registrar su código y body reales. El repositorio exige GET/HEAD, pero hay evidencia previa de una versión publicada que aceptaba POST; vuelve a verificar después de cada despliegue. Una respuesta de salud no acredita todos los permisos ni recorridos de negocio.

5. Compilar e instalar Android

Compilación local

```
flutter build apk --release --dart-define-from-file=.env
# Salida: build/app/outputs/flutter-apk/app-release.apk
Get-FileHash build/app/outputs/flutter-apk/app-release.apk -Algorithm SHA256
```

Antes de distribuir, guarda la revisión Git, la versión de pubspec.yaml, fecha, tamaño y SHA-256 del APK. Instala exactamente el archivo cuya huella registraste. Comprueba que abre, muestra el login y conecta con el proyecto Supabase previsto.

Firma de publicación pendiente
En android/app/build.gradle.kts la variante release usa actualmente signingConfigs.debug. Esto permite una compilación local, pero para una distribución final debe configurarse una clave de firma release y generarse un APK nuevo. No presentes un APK existente como distribución final sin comprobar su firma, huella e instalación.

Instalación en dispositivo

01 Conecta un Android por USB y usa flutter install, o transfiere al teléfono el APK verificado por un canal acordado.

02 En el teléfono abre el APK. Si Android solicita permiso para instalar desde esa fuente, sigue las indicaciones del dispositivo solo para la fuente de confianza.

03 Abre Veterinaria EIVET, inicia sesión con una cuenta ficticia y comprueba el recorrido permitido. Documenta modelo, versión Android, fecha, versión app y captura sin datos sensibles.

6. Publicar el panel web

01 Conecta el repositorio al sitio Netlify autorizado. Comprueba la rama de producción y la revisión que se va a publicar.

02 Configura SUPABASE_URL y SUPABASE_PUBLISHABLE_KEY como variables de build en Netlify para ese proyecto Supabase.

03 El archivo netlify.toml ejecuta bash scripts/build_netlify.sh y publica build/web. El script instala Flutter 3.44.8 y compila en modo release con APP_MODE=supabase.

04 Tras el deploy, abre la URL del panel indicada para el proyecto y comprueba inicio de sesión del personal, lectura de perfiles y acceso a módulos autorizados. Guarda el ID del deploy y capturas.

```
# Configuración del repositorio: netlify.toml
command = bash scripts/build_netlify.sh
publish = build/web
```

Variables del cliente
La publishable key se incorpora al JavaScript web y al APK. La protección de datos depende de Auth y RLS, no del secreto de esa clave. Nunca configures service_role como variable de build que termine dentro de la aplicación.

7. Verificación antes de entregar

Comprobación	Evidencia mínima
Base y RLS	Políticas activas; dos propietarios ficticios: cada uno ve su mascota y no la ajena.
Funciones	GET salud y rechazo de crear-propietario sin sesión/con rol no autorizado.
Android	APK firmado para la distribución prevista; SHA-256; instalación y apertura en un dispositivo.
Web	Deploy Netlify identificado y login del personal con rol confirmado.
Citas	Solicitud ficticia creada y vista por su propietario; cancelación o rechazo documentado.

Si una inserción devuelve 403 por RLS, no se considera completado el recorrido de citas. Conserva respuesta y revisa las políticas desplegadas antes de reintentar. No marques una prueba como aprobada sin su resultado y comprobación posterior.

8. Mantenimiento y referencias

Para cada nueva entrega, conserva revisión Git, artefactos anteriores, huellas y capturas. Antes de modificar SQL de producción, revisa dependencias, datos y posibilidades de restauración. Publica una versión web o APK nueva solo tras repetir las verificaciones esenciales.

Archivos del proyecto

Archivo	Uso
README.md	Resumen, comandos Flutter y despliegue Netlify.
docs/SUPABASE_EIVET.md	Orden SQL, roles, RLS y Edge Functions.
netlify.toml / scripts/build_netlify.sh	Build y publicación web.
supabase/config.toml / supabase/functions	Configuración y código de funciones.
android/app/build.gradle.kts	Identificador, versión y firma Android.

Documentación oficial consultada

Flutter - Build and release an Android app; Flutter - Build and release a web app; Supabase - Deploy to Production; Netlify - Build environment variables.

Manual redactado a partir del repositorio local y documentación oficial consultada el 10/10/2026. Las instrucciones describen el procedimiento previsto; cada despliegue e instalación debe verificarse en su entorno real.